

Figure 4: Illustrative coarse localization example of “table” with two viewpoints. Spaces covered by the viewpoints are voxelized and each voxel is assigned votes based on the number of viewpoints that contain it. Clustering voxels of at least v votes generates proposals at a confidence level of v . Non-maximum suppression is applied to remove redundant proposals.

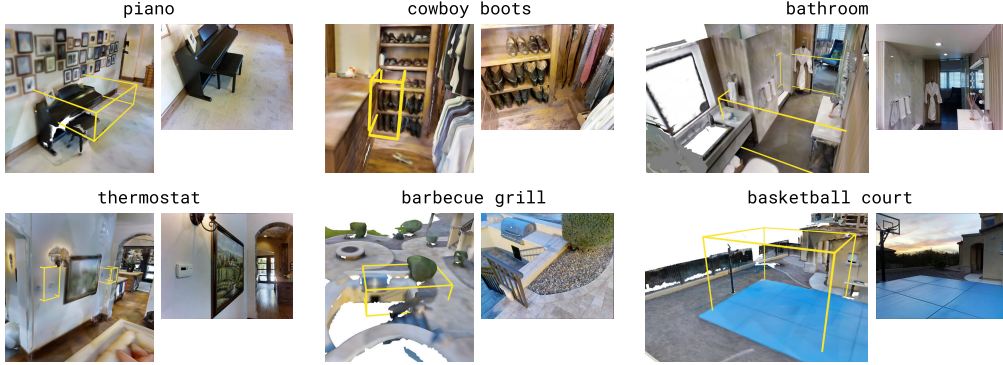


Figure 5: Example coarse localizations for various objects and locations, along with images where they were recognized. The large vocabulary of image tagging models allows our framework to localize uncommon categories such as “thermostat”, “cowboy boots” and “basketball court”.

Localizing a tag begins with retrieving that tag’s corresponding viewpoints and mapping each viewpoint to its 3D frustum using the stored far plane distance. The space covered by the viewpoints is voxelized and each voxel is assigned votes corresponding to the number of viewpoints that contains it. The voxel votes represent a likelihood of the tag over the space. Localization proposals at a confidence level v are extracted by thresholding out voxels below v votes and clustering the remaining voxels using DBSCAN [53]. A confidence level of n indicates that a proposal is consistent across at least n viewpoints. We set a series of clustering thresholds to capture not just the global but also local maxima of the voxel likelihood. Lastly, a non-maximum suppression step is applied to remove proposals that contain within it proposals of higher confidence levels. Fig. 5 shows example coarse localizations for various tags of objects and locations.

3.3 Grounding Large Language Models

We opt for the simple method of grounding the LLM by appending the list of unique tags from the tag map into the prompt of the LLM, as shown in Figure 2. The LLM is additionally given access to an API for requesting information from the tag map. The API exposes the following functions:

- `localize_tag(tag)`: Returns the proposals for the tag along with their confidence levels.
- `region_region_dist(r1, r2)`: Computes the distance between regions $r1$ and $r2$.
- `point_region_dist(p, r)`: Computes the distance to reach region r from point p .

The API enables the LLM to reason spatially about the tags in its prompt. For example, given a query of “go to the kitchen fridge”, the LLM can localize “kitchen” and “fridge” and set the navigation goal to be the fridge proposal with a small distance to a kitchen proposal.

4 Evaluating Coarse-Grained Localizations

The coarse localizations produced by our framework generally do not precisely contain the localized entity as seen in Fig. 5. However, such localizations can still be useful navigation goals for reaching the entities. Evaluating coarse localizations using metrics such as Intersection Over Union (IoU)

fails to assess localizations in terms of “imprecise but still useful” navigation goals. For example, localizing a small area within a bedroom would be a useful goal for navigating to the bedroom but have poor IoU against the room’s bounding box label.

4.1 Metrics for Evaluating Coarse-Grained Localizations

Our goal is to produce a proposal P that is useful for locating the desired entity. In other words, if the robot reaches the region P , it should be close enough to an instance of the desired entity $E \in \mathcal{E}$. This is aligned with the objective of the Habitat ObjectNav Challenge [2] benchmark, where an agent is successful in navigating to a goal object if the distance between the agent and the nearest goal object is below 1.0 m¹ and the object can be viewed with basic yaw and pitch movements of the camera. We quantify the usefulness of P as the expected length of the shortest path an agent must travel from P to reach *any* desired entity instance in the set $\mathcal{E}$. We denote this quantity as the Proposal to Entities Distance (P2E) defined as

$$\text{P2E} = \mathbb{E}_{p \sim P} \left[\min_{E \in \mathcal{E}} d(p, E) \right], \quad (1)$$

where $d(\cdot, \cdot)$ is the shortest path length function. We can evaluate P2E over a set of proposals and classify proposals with P2E under a set threshold (e.g. 1.0 m) as relevant. We refer to the portion of relevant proposals for that threshold as the Precision at Threshold and use it to measure the *precision* of that set of proposals.

Symmetrically, we define the Entity to Proposals Distance (E2P) which evaluates for an entity instance E the shortest expected path length from E to reach any proposal for that entity. For a set of entity instances, we compute the portion with E2P below a threshold for a given set of proposals as a measure of the *recall* for that set of proposals, which we denote as the Recall at Threshold.

The expected shortest path length is difficult to compute exactly so we approximate it by constructing a grid graph over the scene that avoids collisions with the scene geometry, allowing the computation of approximate shortest paths. Nodes of the grid graph are assigned to the proposals and entity instances and the expectation is approximated by averaging over the assigned nodes. Computing the approximations of P2E and E2P is equivalent to computing the Directed Average Hausdorff Distance (DAHD) [54], a metric commonly used in medical image segmentation.

4.2 Evaluation Implementation

Evaluations are performed on the Matterport3D dataset [55]. The evaluation is separated into object and room/region evaluations. Object evaluation is done on the 21 common object classes defined by the Habitat ObjectNav Challenge [2]. For regions, we evaluate all classes in Matterport3D except for “other room”, “junk”, “no label”, “outdoor”, “entryway/foyer/lobby”, and “dining booth”. These classes were ignored as they did not refer to actual rooms or corresponded to ambiguously labeled regions. When evaluating a tag map, each class label is mapped to a list of corresponding tags. Proposals are generated for all the corresponding tags that exist in the tag map and are grouped to form the set of proposals for the class. Unless stated otherwise, we report the precision and recall over all classes by averaging the precision and recall across classes. Further details on the implementation of the evaluation are found in Appendix D.

5 Results

5.1 Comparison with Embedding Based Maps

Tag map localizations are compared against localizations from open vocabulary embedding-based maps using the proposed precision and recall metrics. We chose to compare against OpenScene [23] for 3D segmentation using a point cloud representation with an embedding for each point, and OpenMask3D [25] for 3D instance segmentation which stores 3D instance masks with embeddings

¹In the challenge, another 0.1m is allowed, thus the distance threshold is actually 1.1 m.

for each mask. We query all embeddings with all class labels at once, then assign each embedding a class based on the best matching class following Peng et al. [23]. Another option is to query classes individually and assign embeddings a class if they match above a threshold as done in Lu et al. [26]. However, picking a suitable threshold is difficult and thresholds may not transfer across scenes. For OpenScene, we cluster the segmentation output to obtain instance bounding boxes following Lu et al. [26]. The comparison is only done over the Matterport3D test split as OpenScene uses the train and validation splits during training.

Object Precision-Recall	Precision at Threshold [m]				Recall at Threshold [m]			
	0.1	0.5	1.0	2.0	0.1	0.5	1.0	2.0
OpenScene [23]	0.31	0.36	0.41	0.46	0.12	0.29	0.61	0.88
OpenMask3D [25]	0.29	0.35	0.41	0.48	0.05	0.19	0.41	0.60
Tag Map (Ours)	0.28	0.39	0.46	0.53	0.13	0.42	0.65	0.82

Table 1: Evaluation on object classes against open-vocabulary map representations.

Region Precision-Recall	Precision at Threshold [m]				Recall at Threshold [m]			
	0.5	1.0	2.0	3.0	0.5	1.0	2.0	3.0
OpenScene [23]	0.32	0.33	0.36	0.38	0.22	0.36	0.56	0.64
OpenMask3D [25]	0.56	0.58	0.61	0.63	0.10	0.26	0.49	0.63
Tag Map (Ours)	0.46	0.50	0.54	0.56	0.11	0.22	0.44	0.64

Table 2: Evaluation on region classes against open-vocabulary map representations.

The comparison results are reported in Tables 1 and 2. For objects, the tag map outperforms both embedding-based maps for most precision and recall thresholds. However, the embedding-based maps have better precision when the threshold is set to a smaller value (e.g. 0.1 m), due to these methods storing geometric information to produce tighter instance bounding boxes. For regions, OpenMask3D achieves the best precision across thresholds as it produces smaller instance proposals that are often precisely located within the region label. However, these smaller proposals have lower recall as the larger size of region labels results in higher E2P. Conversely, OpenScene tends to produce much larger proposals for region classes resulting in lower E2P and better recall. The tag map achieves good precision for region classes but the worst recall compared to the other methods. Note that larger precision-recall thresholds are used for the regions as region labels are much larger than object labels. We also compared the memory usage of the maps in Fig. 6. The tag map uses much less memory than the embedding-based maps as storing the embedding vectors requires significantly more memory than text-based tags. The comparison is especially stark against OpenScene which stores per-point embeddings for a scene point cloud.

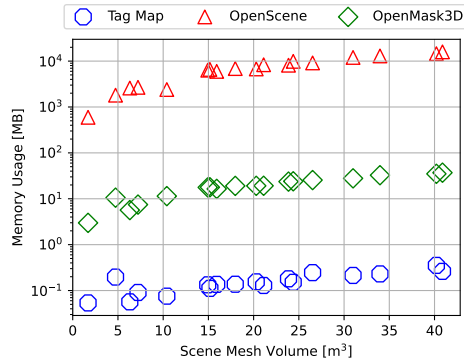


Figure 6: Memory use of the tag map and open-vocabulary maps across scenes of various sizes. Scene size is measured as the voxel volume after voxelizing the scene mesh

5.2 Comparison with CLIP Viewpoint Retrieval

The recent work of Chang et al. [5] proposed to use store viewpoint CLIP embeddings and to search over the embeddings when given an object query. We modify our coarse localization method to retrieve viewpoints using CLIP instead of using the relevant tags. To make the comparison fair, for both methods, we retrieve only the top K most confident viewpoints. For tags, we use the tag prediction confidences from the tagging model. We also include the unmodified tag localization method as a baseline, denoted as “all views”. Precision and recall comparisons for the two methods at different thresholds are reported in Fig. 7. Retrieval via tags consistently outperforms CLIP